DOCUMENTO DE IMPLEMENTAÇÃO · OUTUBRO DE 2026

Blueprint Técnico

Este documento entrega tudo que é preciso para colocar a fase 1 do Narrador em pé: o escopo real, a arquitetura, o modelo de dados, o protocolo entre mestre e motor, o adaptador do serviço de linguagem e o roteiro em cards, na ordem em que devem ser executados. Complementa o Mapa do Projeto e os Casos de Uso.

ESTADO DESTA VERSÃO

Entrega 2: capítulos 1 a 6. O esquema de dados executado com provas, as três máquinas de estado, o protocolo entre mestre e motor e o adaptador do serviço de linguagem. Fluxos, cards, testes e fechamento entram nas entregas 3 e 4.

6	49	5	9
capítulos nesta entrega	casos de uso da fase 1 cobertos	pacotes do código	escolhas de tecnologia propostas

Para quem escrevemos isto

Para a pessoa que vai criar o repositório e precisa saber exatamente o que construir primeiro. Cada capítulo responde a uma pergunta prática: qual é o escopo real, como o dado é modelado, como o mestre fala com o motor, como falamos com o serviço de linguagem e qual card pegar em seguida. Nada aqui é sugestão de arquitetura para discutir depois: é o que vamos construir, salvo o que está marcado como proposta.

A regra que organiza todo o documento

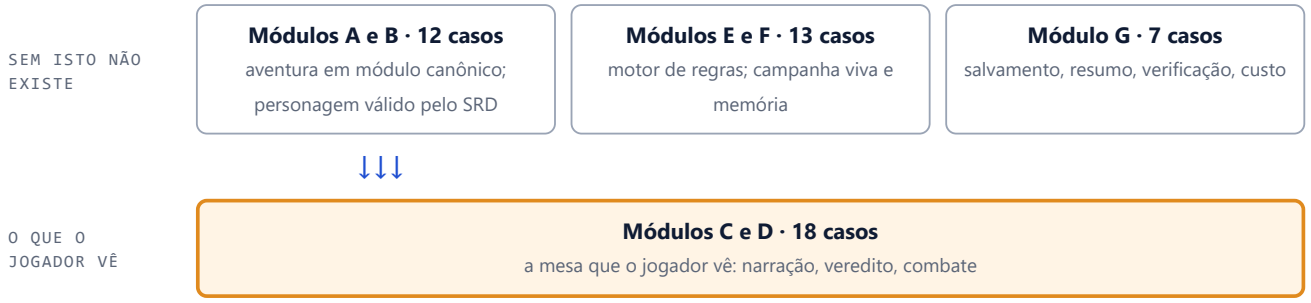
Todo dado é do motor. Todo fato, veredito e desvio deixa rastro consultável. O serviço de linguagem nunca grava estado: ele lê um recorte, propõe e narra, e o motor decide. Se um card contrariar isso, o card está errado.

Narrador · mestre por inteligência artificial para D&D 5e, regras de 2024, SRD 5.2 em CC-BY-4.0 · acompanha o Mapa do Projeto (revisão 4) e os Casos de Uso (56 fichas)

CAPÍTULO 1

O escopo real da primeira entrega

A fase 1 tem 49 casos de uso e todos entram, mas não todos de uma vez. O primeiro código entregue é o esqueleto ambulante: a fatia mais fina de ponta a ponta, jogável, que prova a arquitetura antes de qualquer funcionalidade ganhar corpo.



O que a mesa arrasta junto. Colocar só a narração e o combate no ar resultaria numa mesa que inventa regra, esquece o que aconteceu e perde a sessão na primeira queda. Os módulos A, B, E, F e G sustentam C e D e entram juntos.

O ESQUELETO AMBULANTE, A PRIMEIRA COISA JOGÁVEL

Um personagem pré-gerado (B02), uma aventura original mínima já no formato canônico, consultada pelo mestre (A05), uma cena narrada (C03), uma tentativa com veredito e rolagem (C05, E01, E02), um encontro com iniciativa, turnos, ataque e fim (D01, D02, D03, D04, D08), tudo gravado na campanha viva (F01, G01) e retomável (C02, C09). No terminal, em português. Nada de importar PDF, criar personagem ou subir de nível ainda: isso vem card a card depois que a espinha dorsal estiver provada e jogada pelo cliente.

Os 49 casos de uso da fase 1, por módulo

MÓDULO	CASOS	POR QUE ENTRA AGORA
A · Campanha e módulo	A01 a A07	Sem módulo canônico não há gabarito; é a tese do produto.
B · Personagem	B01 a B05	Sem ficha válida o motor não tem com o que calcular.
C · Mesa e narração	C01 a C10	É o que o jogador vê; inclui o veredito da regra RN-11.
D · Combate	D01 a D08	A parte mais exigente do motor, por zonas na fase 1 (DC-13).
E · Regras e dados	E01 a E06	O motor determinístico e o SRD como dado; nada roda sem ele.
F · Campanha viva	F01 a F06	Fatos, desvios, caminho alternativo, retomada, revisão.
G · Processos automáticos	G01 a G07	Salvamento a cada fato, consistência, recusa do serviço, atribuição, custo.

O QUE FICA DE FORA, DE PROPÓSITO

Grupo (H01 a H03), campanha livre (I01 a I03) e memória de campanha longa (F07) são das fases 2 e 3. Interface web, grade tática de combate (decisão DC-13), voz e mapa também. Tudo isso está desenhado nos Casos de Uso e no Mapa, e o modelo de dados deste documento já foi pensado para recebê-los sem migração destrutiva: a campanha viva tem lugar para mais de um jogador, e o módulo canônico gerado é o mesmo objeto do importado.

CAPÍTULO 2

A arquitetura que vamos construir

Um processo só na fase 1, cinco pacotes com fronteiras claras e um lugar onde o dado mora. Quem vem de fora entra por uma porta, e nenhuma delas conversa com o dado sem passar pelo motor, onde vivem as regras.



As cinco camadas do sistema. O serviço de linguagem aparece na primeira camada porque, para nós, ele é um usuário como qualquer outro: recebe um recorte, pede coisas pelo protocolo e recebe respostas. A diferença é que ele nunca grava estado.

Um processo, cinco pacotes

PROCESSO

`narrador`

O programa de terminal. Abre a campanha, roda a sessão, mostra telas de texto. Único processo da fase 1; a web da fase 2 é outro processo sobre os mesmos pacotes.

PACOTE

`motor`

Regras puras: dados com semente, testes, ficha e derivados, combate por zonas, condições, validação de pedido. Sem rede, sem disco, sem relógio de parede. É onde moram os testes que não podem falhar.

PACOTE

`srd`

O SRD 5.2 em formato estruturado, carregado do PDF oficial com o acelerador conferido, busca por nome (inglês, português, nomes de 2014) e o texto de atribuição.

PACOTE

`modulo`

Importação do arquivo, extração de texto, estruturação com ou sem serviço, conferência, mapeamento para o SRD, versões imutáveis do módulo canônico.

PACOTE

`campanha`

Campanha viva: diário só de inserção de fatos, desvios, vereditos e rolagens; instantâneos; retomada; o que o personagem sabe.

PACOTE

`mestre`

Montagem do contexto por cena, protocolo de ferramentas, verificação de consistência e vazamento, adaptador do serviço de linguagem, classificação de falhas, medição de custo.

POR QUE SEPARAR O MOTOR DO MESTRE

O mestre é probabilístico e externo: pode errar, recusar, cair. O motor é determinístico e local: dado o mesmo estado e a mesma semente, produz o mesmo resultado, sempre. Juntar os dois significaria não poder testar regra sem serviço de linguagem, nem auditar rolagem sem reler narração. Separando, o motor roda inteiro nos testes sem rede, e o mestre pode ser trocado por configuração sem tocar numa regra.

As escolhas de tecnologia, como proposta proposta

Proposta para o cliente bater o martelo. O critério, em ordem: o que o Mapa já comprometeu, o que os casos de uso exigem, o que já existe nesta máquina, e o que o cliente já domina em outros projetos. Nenhuma linha abaixo nomeia provedor nem modelo de linguagem: isso é decidido por medição, no card da bancada.

ESCOLHA	O QUE PROPOMOS	POR QUÊ
Linguagem	Python 3.13	O Mapa comprometeu o motor testado isolado com pytest. A extração de PDF, a validação de dados e a chamada ao serviço já têm biblioteca instalada nesta máquina. O cliente já mantém uma API em Python. **Alternativa considerada:** TypeScript com Node 22, como no outro projeto do cliente: mesma ferramenta para a web da fase 2 e o mesmo fluxo de integração contínua que ele já opera.
Validação e tipos	pydantic 2 com tipagem estrita conferida por mypy	Ficha, módulo canônico, fatos e mensagens do protocolo são dados com forma fixa; validar na borda e tipar por dentro evita a classe inteira de erro de regra por campo errado. **Alternativa considerada:** dataclasses da biblioteca padrão com validação à mão: menos dependência, mais código de conferência escrito e testado por nós.
Dado da campanha viva	SQLite, um arquivo por campanha, diário só de inserção garantido por gatilho	Local por regra (RN-09), atômico e retomável por regra (RN-10), consultável para auditoria (E06, F06) sem carregar tudo em memória. A prova dos gatilhos entra na entrega 2. **Alternativa considerada:** Arquivos de texto em linhas (uma linha por fato): mais simples de inspecionar à mão, sem consulta estruturada e com atomicidade a cargo nosso.
SRD estruturado	PDF oficial 5.2.1 (CC-BY-4.0) como fonte de verdade; o conjunto de dados aberto da comunidade para 2024 como acelerador, conferido registro a registro contra o PDF	O acelerador tem lacunas conhecidas (magias sem texto, itens faltando) e a licença dos arquivos derivados não é declarada; a conferência contra o PDF fecha a lacuna LC-01 com prova. **Alternativa considerada:** Conversões em Markdown da comunidade: mais fáceis de ler, com divergências relatadas em criaturas e itens.
Extração de texto de PDF	PyMuPDF	Já instalado, rápido, preserva a página de origem por bloco, que o caso de uso A01 exige. **Alternativa considerada:** pypdf: puro Python, mais lento, também instalado.
Serviço de linguagem	Adaptador por interface, com uma implementação por provedor usando o SDK oficial dele, e um provedor falso para testes	O provedor é escolhido pela bancada do card correspondente, por custo, latência, qualidade em português e comportamento nas cenas da decisão DC-11. Nada no código fora do adaptador sabe qual é. **Alternativa considerada:** Chamadas HTTP diretas: menos dependência, mais código de protocolo a manter a cada mudança do provedor.

ESCOLHA	O QUE PROPOMOS	POR QUÊ
Terminal	Biblioteca de terminal rica para cor e painéis, sobre entrada de texto simples	A fase 1 é texto; a leitura da mesa melhora com cor e blocos, sem exigir interface gráfica. **Alternativa considerada:** Só a biblioteca padrão: zero dependência, telas mais pobres.
Testes	pytest, com testes de propriedade para o motor	Comprometido no Mapa. Testes de propriedade cobrem faixas de dado, limites de regra e transições inválidas melhor do que exemplos escritos à mão. **Alternativa considerada:** Só exemplos com pytest: suficiente no início, pior para o motor.
Qualidade e integração contínua	ruff para lint e formato, mypy para tipos, repositório privado com integração contínua a cada push	Mesmo esquema de entrega dos outros projetos do cliente: a integração verde é o registro de entrega de cada card. **Alternativa considerada:** Sem integração contínua: cada card provado à mão, sem histórico.

CAPÍTULO 3

O dado, tabela por tabela

Dois arquivos SQLite por campanha jogada: um para a campanha viva e um para a aventura de onde ela vem. Dezessete tabelas, quinze gatilhos. Antes do primeiro CREATE TABLE, sete decisões que valem para todas elas.

Só inserção no diário

Fato, rolagem, veredito e desvio nunca são alterados nem apagados. Correção é registro novo apontando para o antigo. O banco recusa, não o código.

Versão publicada é imutável

Um módulo canônico publicado não muda; só nasce versão nova. O banco recusa inserção, alteração e exclusão em versão publicada.

Tempo em UTC, texto ISO

Toda data é texto ISO 8601 em UTC. O relógio do mundo do jogo é outro dado, no estado da campanha, e nunca se confunde com o relógio real.

Estado é restrição do banco

Tipo de registro, conhecimento do personagem, origem do módulo, classificação da mesa: todos CHECK no banco. Gravar valor inexistente é erro, não bug descoberto depois.

Nada é apagado

Remover aventura (A07) apaga o arquivo inteiro, não linhas. Dentro de um arquivo, nada é excluído; o que deixa de valer recebe sucessor ou marca.

A regra vive no banco quando pode

Dificuldade de teste na escala do SRD, segredo com condição de descoberta, mapeamento sem SRD só se pendente: tudo CHECK. O motor não precisa acertar sozinho.

Um arquivo, uma unidade de posse

A aventura e tudo derivado dela ficam no mesmo arquivo, ao lado do original do dono da mesa. Copiar a pasta é a cópia de segurança; apagar a pasta cumpre a regra RN-09.

ESTE ESQUEMA FOI EXECUTADO, NÃO SÓ ESCRITO

Todo o SQL das próximas páginas rodou num SQLite 3.50.4 limpo (Python 3.13.14) antes de entrar neste documento: 10 tabelas e 10 gatilhos na campanha viva, 7 tabelas e 5 gatilhos na aventura, criados sem erro. A semente foi aplicada e as 15 provas do fim do capítulo são a saída real do banco. O roteiro que faz isso fica no repositório e roda na integração contínua.

As dezessete tabelas e para que servem

TABELA	ARQUIVO	GUARDA
`campanha`	campanha	A campanha: aventura e versão fixadas, semente das rolagens, classificação, tom e configuração da mesa (C01, C10).
`personagem`	campanha	A ficha, versionada: cada subida de nível ou mudança de regra é uma versão nova; a anterior fica (B01, B04).
`sessao`	campanha	Abertura, encerramento e motivo de fim de cada sessão (C02, C09, F05).
`diario`	campanha	Tudo o que aconteceu, em ordem, com o caso de uso que gravou. Só inserção (G01).
`rolagem`	campanha	O diário de dados: contador no gerador da campanha, expressão, cada dado, composição, total, motivo (E01, E06).
`fato`	campanha	O estado do mundo: tipo, item canônico afetado, conhecimento do personagem e o estado anterior que sucede (F01, F04).

TABELA	ARQUIVO	GUARDA
desvio	campanha	O que o módulo dizia, o que foi feito, o motivo, a regra invocada e a marca de revisão (F02, F06).
veredito	campanha	A situação comparável, o tipo, a perícia e a dificuldade, se foi pergunta ou ação, e a divergência apurada (C04, C05, G04).
instantaneo	campanha	Estado consolidado até um registro do diário, para abrir rápido; o diário segue sendo a verdade (G01, C02).
regra_de_mesa	campanha	Instruções do dono da mesa dadas na revisão, aplicadas ao mestre dali em diante (F06).
aventura	aventura	Título, caminho local do arquivo, impressão digital, formato, edição informada (A01).
texto_pagina	aventura	O texto extraído, página a página, para a referência de origem de cada item (A01, A02).
autorizacao_envio	aventura	Cada autorização, concedida ou negada, de enviar o texto ao serviço de linguagem (DC-12, A02).
modulo_versao	aventura	As versões do módulo canônico: rascunho ou publicada, origem importada ou gerada (A03, A06, I02).
item_canonico	aventura	Cenas, locais, personagens, criaturas, itens, ganchos e segredos, com página de origem e marcas de incerto e segredo (A02, A03).
ligacao	aventura	O grafo: o que leva a quê, quem sabe de quê, o que contém o quê, e o caminho crítico (A03, A05, F03).
mapeamento_srd	aventura	Criatura ou item para ficha do SRD, ficha própria do dono ou pendente, com a versão do SRD (A04).

CAMPANHA VIVA: MIGRAÇÃO 0001, PRONTA PARA RODAR

```
-- Campanha viva: um arquivo SQLite por campanha. Migração 0001.
-- Regra do arquivo: o diário só recebe inserção. Quem tentar alterar ou apagar
-- recebe erro do banco, não aviso do código.
pragma foreign_keys = on;
pragma journal_mode = wal;

create table campanha (
  id              text primary key,                -- uuid
  nome            text not null,
  aventura_id     text not null,                   -- id na pasta de aventuras
  modulo_versao   integer not null,                -- versão fixada (C01)
  semente         text not null,                   -- semente das rolagens (E01)
  classificacao   text not null check (classificacao in ('livre','14','16','adulto')),
  tom             text not null default 'serio',
  configuracao    text not null default '{}',      -- json: dado físico, progressão, regras opcionais, limite de
custo (C10)
  criada_em       text not null default (strftime('%Y-%m-%dT%H:%M:%fZ','now'))
);

-- Ficha versionada: cada subida de nível ou mudança de regra é uma versão nova (B01, B04).
create table personagem (
  id              text not null,
  versao          integer not null,
  campanha_id     text not null references campanha (id),
  nome            text not null,
  origem          text not null check (origem in ('criado','pre_gerado')),
  ficha           text not null,                   -- json validado pelo motor
  srd_versao      text not null,
  criado_em       text not null default (strftime('%Y-%m-%dT%H:%M:%fZ','now')),
  primary key (id, versao)
);

create table sessao (
  id              text primary key,
  campanha_id     text not null references campanha (id),
```

```
  numero          integer not null,
  aberta_em       text not null default (strftime('%Y-%m-%dT%H:%M:%fZ','now')),
  encerrada_em    text,
  motivo_fim      text check (motivo_fim in ('encerrada','interrompida','retomada')),
  unique (campanha_id, numero)
);

-- O diário: toda coisa que aconteceu, em ordem, só inserção (G01).
create table diario (
  seq             integer primary key autoincrement,
  campanha_id     text not null references campanha (id),
  sessao_id       text references sessao (id),
  tipo            text not null check (tipo in (
                    'fato','desvio','veredito','rolagem','consulta','verificacao',
                    'recusa','configuracao','sessao','resumo','revisao','custo','ocorrencia')),
  cena_id         text,                           -- item canônico da cena, se houver
  origem          text not null,                  -- caso de uso que gravou (C05, D04...)
  dados           text not null,                  -- json com o conteúdo do registro
  registrado_em   text not null default (strftime('%Y-%m-%dT%H:%M:%fZ','now'))
);
create index diario_por_campanha on diario (campanha_id, seq);
create index diario_por_tipo     on diario (campanha_id, tipo, seq);

create trigger diario_sem_update before update on diario
begin select raise(abort, 'diario_somente_insercao'); end;
create trigger diario_sem_delete before delete on diario
begin select raise(abort, 'diario_somente_insercao'); end;

-- Rolagens: o diário de dados (E01). O contador é a posição no gerador da campanha.
create table rolagem (
  seq             integer primary key references diario (seq),
  campanha_id     text not null references campanha (id),
  contador        integer,                        -- nulo quando informada (dado físico)
  expressao       text not null,                  -- 1d20+3
  modo            text not null check (modo in ('normal','vantagem','desvantagem')),
  dados           text not null,                  -- json: cada dado
  modificadores   text not null,                  -- json: composição
  total           integer not null,
  motivo          text not null,
  informada       integer not null default 0 check (informada in (0,1)),
  unique (campanha_id, contador)
);
create trigger rolagem_sem_update before update on rolagem
begin select raise(abort, 'diario_somente_insercao'); end;
create trigger rolagem_sem_delete before delete on rolagem
begin select raise(abort, 'diario_somente_insercao'); end;

-- Fatos: o estado do mundo, com conhecimento do personagem e encadeamento (F01, F04).
create table fato (
  seq             integer primary key references diario (seq),
  campanha_id     text not null references campanha (id),
  tipo_fato       text not null check (tipo_fato in (
                    'estado_local','estado_personagem','posse','relacao','conhecimento','evento')),
  referencia      text,                           -- item canônico afetado
  conhecimento    text not null check (conhecimento in ('conhecido','oculto','acredita')),
  substitui_seq   integer references fato (seq),  -- estado anterior que este sucede; nunca apagado
  resumo          text not null
);
create trigger fato_sem_update before update on fato
begin select raise(abort, 'diario_somente_insercao'); end;
create trigger fato_sem_delete before delete on fato
begin select raise(abort, 'diario_somente_insercao'); end;

-- Desvios do canônico, com motivo e regra invocada (F02).
create table desvio (
  seq             integer primary key references diario (seq),
  campanha_id     text not null references campanha (id),
  item_canonico   text,                           -- o que o módulo dizia
  o_que_mudou     text not null,
  motivo          text,                           -- nulo = "sem motivo informado", elevado na revisão
```

```
  regra          text,                          -- RN-02, RN-03...
  revisado       integer not null default 0 check (revisado in (0,1,2))  -- 0 pendente, 1 aprovado, 2 ruim
);
-- revisado é a única coluna que muda, e só de 0 para 1 ou 2 (F06): o passado não muda, a marca sim.
create trigger desvio_so_marca before update on desvio
when new.seq <> old.seq or new.item_canonico is not old.item_canonico or new.o_que_mudou <> old.o_que_mudou
  or new.motivo is not old.motivo or new.regra is not old.regra or old.revisado <> 0
begin select raise(abort, 'desvio_so_marca_de_revisao'); end;
create trigger desvio_sem_delete before delete on desvio
begin select raise(abort, 'diario_somente_insercao'); end;

-- Vereditos, para a consistência (C04, C05, G04).
create table veredito (
  seq            integer primary key references diario (seq),
  campanha_id    text not null references campanha (id),
  situacao       text not null,                 -- descrição comparável
  tipo           text not null check (tipo in ('de_cara','impossivel','teste')),
  pericia        text,
  dificuldade    integer check (dificuldade is null or dificuldade between 5 and 30),
  pergunta       integer not null default 0 check (pergunta in (0,1)),  -- 1 = "e se eu fizer?" (C04)
  rolagem_seq    integer references rolagem (seq),
  divergiu_de    integer references veredito (seq),
  motivo_divergencia text
);
create trigger veredito_sem_update before update on veredito
begin select raise(abort, 'diario_somente_insercao'); end;
create trigger veredito_sem_delete before delete on veredito
begin select raise(abort, 'diario_somente_insercao'); end;

-- Instantâneo do estado para abertura rápida; o diário continua sendo a verdade (G01, C02).
create table instantaneo (
  campanha_id    text not null references campanha (id),
  ate_seq        integer not null,
  estado         text not null,                 -- json do estado consolidado
  gravado_em     text not null default (strftime('%Y-%m-%dT%H:%M:%fZ','now')),
  primary key (campanha_id, ate_seq)
);

-- Regras de mesa dadas na revisão (F06), aplicadas ao mestre dali em diante.
create table regra_de_mesa (
  id             integer primary key autoincrement,
  campanha_id    text not null references campanha (id),
  texto          text not null,
  origem_seq     integer references diario (seq),
  ativa          integer not null default 1 check (ativa in (0,1)),
  criada_em      text not null default (strftime('%Y-%m-%dT%H:%M:%fZ','now'))
);
```

PASTA DE AVENTURAS: MIGRAÇÃO 0001, PRONTA PARA RODAR

```
-- Pasta de aventuras: um arquivo SQLite por aventura importada, ao lado do arquivo original.
-- Regra do arquivo: versão publicada do módulo canônico é imutável; só nasce versão nova.
pragma foreign_keys = on;

create table aventura (
  id                  text primary key,
  titulo              text not null,
  origem_arquivo      text not null,             -- caminho local, nunca copiado para fora (RN-09)
  impressao_digital   text not null unique,      -- hash do conteúdo (A01)
  formato             text not null check (formato in ('pdf','markdown','texto')),
  edicao_informada    text check (edicao_informada in ('2014','2024')),
  texto_parcial       integer not null default 0 check (texto_parcial in (0,1)),
  uso_pessoal_declarado integer not null default 1,
  importada_em        text not null default (strftime('%Y-%m-%dT%H:%M:%fZ','now'))
);

create table texto_pagina (
  aventura_id         text not null references aventura (id),
  pagina              integer not null,
  texto               text not null,
```

```
  primary key (aventura_id, pagina)
);

-- Autorização de envio ao serviço de linguagem, registrada por importação (DC-12, A02).
create table autorizacao_envio (
  id                  integer primary key autoincrement,
  aventura_id         text not null references aventura (id),
  concedida           integer not null check (concedida in (0,1)),
  trechos             integer not null,           -- quantos trechos foram mostrados
  registrada_em       text not null default (strftime('%Y-%m-%dT%H:%M:%fZ','now'))
);

-- Versões do módulo canônico (A03, A06). publicada_em nulo = rascunho.
create table modulo_versao (
  aventura_id         text not null references aventura (id),
  versao              integer not null,
  origem              text not null check (origem in ('importado','gerado','gerado_revisado')),
  publicada_em        text,
  impressao_texto     text not null,              -- hash do texto de origem no momento
  primary key (aventura_id, versao)
);

create table item_canonico (
  id                  text not null,
  aventura_id         text not null,
  versao              integer not null,
  tipo                text not null check (tipo in (
                        'cena','local','personagem','criatura','item','gancho','segredo')),
  nome                text not null,
  dados               text not null,              -- json
  pagina_origem       integer,
  incerto             integer not null default 0 check (incerto in (0,1)),
  segredo             integer not null default 0 check (segredo in (0,1)),
  condicao_descoberta text,                       -- obrigatória quando segredo = 1
  primary key (id, aventura_id, versao),
  foreign key (aventura_id, versao) references modulo_versao (aventura_id, versao),
  check (segredo = 0 or condicao_descoberta is not null)
);

create table ligacao (
  aventura_id         text not null,
  versao              integer not null,
  de_item             text not null,
  para_item           text not null,
  tipo                text not null check (tipo in ('leva_a','contem','sabe_de','possui','gatilho')),
  caminho_critico     integer not null default 0 check (caminho_critico in (0,1)),
  primary key (aventura_id, versao, de_item, para_item, tipo),
  foreign key (aventura_id, versao) references modulo_versao (aventura_id, versao)
);

-- Mapeamento de criaturas e itens para o SRD (A04).
create table mapeamento_srd (
  aventura_id         text not null,
  versao              integer not null,
  item_id             text not null,
  srd_id              text,                       -- nulo com ficha própria ou pendente
  srd_versao          text not null,
  ficha_propria       text,                       -- json, fica no módulo do dono, fora do produto
  pendente            integer not null default 0 check (pendente in (0,1)),
  fora_da_escala      integer not null default 0 check (fora_da_escala in (0,1)),
  primary key (aventura_id, versao, item_id),
  foreign key (aventura_id, versao) references modulo_versao (aventura_id, versao),
  check (pendente = 1 or srd_id is not null or ficha_propria is not null)
);

-- Imutabilidade: nada de uma versão publicada muda; só nasce versão nova (A06).
create trigger item_publicado_sem_update before update on item_canonico
when exists (select 1 from modulo_versao v where v.aventura_id = old.aventura_id and v.versao = old.versao and
v.publicada_em is not null)
begin select raise(abort, 'versao_publicada_imutavel'); end;
```

```
create trigger item_publicado_sem_delete before delete on item_canonico
when exists (select 1 from modulo_versao v where v.aventura_id = old.aventura_id and v.versao = old.versao and
v.publicada_em is not null)
begin select raise(abort, 'versao_publicada_imutavel'); end;
create trigger item_publicado_sem_insert before insert on item_canonico
when exists (select 1 from modulo_versao v where v.aventura_id = new.aventura_id and v.versao = new.versao and
v.publicada_em is not null)
begin select raise(abort, 'versao_publicada_imutavel'); end;
create trigger ligacao_publicada_sem_mudanca before insert on ligacao
when exists (select 1 from modulo_versao v where v.aventura_id = new.aventura_id and v.versao = new.versao and
v.publicada_em is not null)
begin select raise(abort, 'versao_publicada_imutavel'); end;
create trigger versao_publicada_fixa before update on modulo_versao
when old.publicada_em is not null
begin select raise(abort, 'versao_publicada_imutavel'); end;
```

POR QUE DOIS ARQUIVOS E NÃO UM BANCO

A aventura é obra de terceiro e propriedade do dono da mesa; a campanha é o que aconteceu na mesa dele. Separados, a aventura pode ser removida inteira (A07) sem tocar na campanha, que vira só leitura; uma mesma aventura serve várias campanhas; e a campanha carrega a versão exata do módulo que usou, não uma cópia. Na fase 2, com mais de um jogador, a campanha continua um arquivo: o que muda é quem escreve nela, não onde ela mora.

SEMENTE: A AVENTURA ORIGINAL MÍNIMA, NO FORMATO CANÔNICO

```
-- Semente da pasta de aventuras: a aventura original mínima do projeto, "O Porão da Taverna",
-- já no formato canônico, para testes automáticos e para o esqueleto ambulante (DC-09).
insert into aventura (id, titulo, origem_arquivo, impressao_digital, formato, edicao_informada)
values ('av-porao', 'O Porão da Taverna', 'semente/porao-da-taverna.md', 'sha256:semente-porao-v1', 'markdown',
'2024');

-- Nasce como rascunho; é publicada no fim, depois de conferida (A03). Publicada, não muda mais.
insert into modulo_versao (aventura_id, versao, origem, publicada_em, impressao_texto)
values ('av-porao', 1, 'importado', null, 'sha256:semente-porao-v1');

insert into item_canonico (id, aventura_id, versao, tipo, nome, dados, pagina_origem) values
 ('cena-taverna',  'av-porao', 1, 'cena',   'A taverna ao anoitecer',
   '{"descricao":"Mesas vazias, um taverneiro nervoso e um alçapão trancado atrás do balcão.","objetivo":"descobrir o
que faz barulho no porão"}', 1),
 ('cena-porao',    'av-porao', 1, 'cena',   'O porão',
   '{"descricao":"Escuro, úmido, barris roídos. Dois pares de olhos vermelhos.","encontro":["cri-rato-1","cri-rato-
2"]}', 2),
 ('cena-tunel',    'av-porao', 1, 'cena',   'O túnel atrás dos barris',
   '{"descricao":"Um túnel recém-cavado leva ao esgoto. No chão, um anel de prata.","objetivo":"recuperar o anel do
taverneiro"}', 2),
 ('loc-taverna',   'av-porao', 1, 'local',  'Taverna do Cavalo Manco', '{"zonas":["salão","balcão","alçapão"]}', 1),
 ('np-taverneiro', 'av-porao', 1, 'personagem', 'Taverneiro',
   '{"objetivo":"recuperar o anel sem admitir que o perdeu no jogo","sabe":["o barulho começou há três
noites"],"atitude":"nervoso"}', 1),
 ('cri-rato-1',    'av-porao', 1, 'criatura', 'Rato gigante', '{"quantidade":1}', 2),
 ('cri-rato-2',    'av-porao', 1, 'criatura', 'Rato gigante', '{"quantidade":1}', 2),
 ('it-anel',       'av-porao', 1, 'item',   'Anel de prata do taverneiro', '{"valor_po":25}', 2),
 ('gancho-barulho','av-porao', 1, 'gancho', 'O barulho no porão', '{"texto":"O taverneiro oferece uma noite grátis a
quem olhar o porão."}', 1);

insert into item_canonico (id, aventura_id, versao, tipo, nome, dados, pagina_origem, segredo, condicao_descoberta)
values
 ('seg-anel', 'av-porao', 1, 'segredo', 'O taverneiro perdeu o anel apostando com o cavador do túnel',
   '{"texto":"O túnel foi cavado por um contrabandista que ganhou o anel no jogo e o deixou cair."}', 2, 1,
   'Persuadir ou intimidar o taverneiro, ou examinar as marcas de ferramenta no túnel');

insert into ligacao (aventura_id, versao, de_item, para_item, tipo, caminho_critico) values
 ('av-porao', 1, 'cena-taverna', 'cena-porao', 'leva_a', 1),
 ('av-porao', 1, 'cena-porao',   'cena-tunel', 'leva_a', 1),
 ('av-porao', 1, 'loc-taverna',  'cena-taverna', 'contem', 0),
 ('av-porao', 1, 'np-taverneiro','seg-anel', 'sabe_de', 0),
 ('av-porao', 1, 'cena-tunel',   'it-anel', 'contem', 0);

insert into mapeamento_srd (aventura_id, versao, item_id, srd_id, srd_versao) values
```

```
 ('av-porao', 1, 'cri-rato-1', 'srd:creature:giant-rat', '5.2.1'),
 ('av-porao', 1, 'cri-rato-2', 'srd:creature:giant-rat', '5.2.1');

-- Conferida: publica a versão 1.
update modulo_versao set publicada_em = strftime('%Y-%m-%dT%H:%M:%fZ','now') where aventura_id = 'av-porao' and versao
= 1;
```

As 15 provas que rodamos

O QUE TENTAMOS FAZER	O QUE O BANCO RESPONDEU
Alterar um registro do diário (update diario set dados = ...)	`IntegrityError: diario_somente_insercao`
Apagar um registro do diário (delete from diario)	`IntegrityError: diario_somente_insercao`
Gravar uma segunda rolagem com o mesmo contador da campanha	`IntegrityError: UNIQUE constraint failed: rolagem.campanha_id, rolagem.contador`
Gravar uma rolagem informada em dado físico (contador nulo)	`ok`
Marcar um desvio como revisado (update desvio set revisado = 1)	`ok`
Reescrever o motivo de um desvio já registrado	`IntegrityError: desvio_so_marca_de_revisao`
Criar campanha sem semente de rolagens	`IntegrityError: NOT NULL constraint failed: campanha.semente`
Veredito de teste com dificuldade 45, fora da escala do SRD	`IntegrityError: CHECK constraint failed: dificuldade is null or dificuldade between 5 and 30`
Aplicar a semente: a aventura original mínima, versão 1 publicada	`ok: 10 itens canônicos e 5 ligações gravados`
Alterar um item de uma versão publicada do módulo	`IntegrityError: versao_publicada_imutavel`
Acrescentar uma cena a uma versão publicada	`IntegrityError: versao_publicada_imutavel`
Gravar um segredo sem condição de descoberta	`IntegrityError: CHECK constraint failed: segredo = 0 or condicao_descoberta is not null`
Acrescentar uma cena ao rascunho da versão 2, ainda não publicada	`ok`
Mapear uma criatura sem SRD, sem ficha própria e sem marca de pendente	`IntegrityError: CHECK constraint failed: pendente = 1 or srd_id is not null or ficha_propria is not null`
Percorrer o caminho crítico da semente a partir da primeira cena	`ok: cena-taverna -> cena-porao -> cena-tunel`

A CONTA DA ROLAGEM, CONFERIDA

Uma rolagem de 1d20+3 com semente da campanha e contador 1 produz sempre o mesmo dado, porque o gerador é inicializado com a semente e avançado até o contador. O registro guarda os dois, mais cada dado e a composição do modificador; refazer a rolagem (E06) é inicializar de novo e comparar. Dado físico informado não tem contador, e o banco aceita o nulo só com a marca de informada.

CAPÍTULO 4

As três máquinas de estado

Sessão, combate e tentativa. Cada uma é uma função pura que recebe o estado atual e um evento e devolve o novo estado com os efeitos a executar. Nenhum outro lugar do código muda esses estados. Se alguém precisar mudar, muda aqui, com teste.



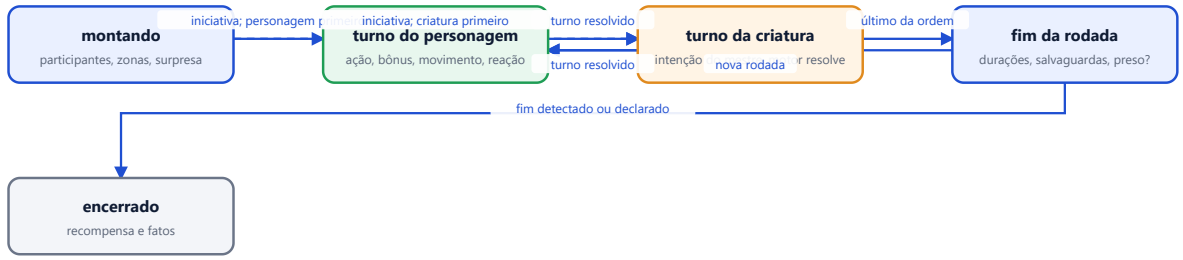
A sessão. A interrupção não é estado: é o arquivo ficar em 'em cena' ou 'em combate' sem encerramento, e a abertura seguinte retomar de lá (F05). 'Pausada' é a mesa parada à espera de serviço ou disco, com tudo salvo.

A tabela de transições da sessão, inteira

Qualquer par fora desta tabela é recusado com o erro transicao_invalida e nada é gravado. Cada linha vira um teste; cada par ausente vira um teste negativo.

ESTADO ATUAL	EVENTO	NOVO ESTADO	O QUE ACONTECE JUNTO
fechada	abrir	abrindo	Carrega instantâneo e diário posterior; detecta sessão não encerrada (F05).
abrindo	contexto_pronto	em_cena	Grava 'sessão N aberta'; o mestre recapitula (C02).
abrindo	contexto_pronto_em_combate	em_combate	Retomada no meio de um combate: reconstrói rodada, ordem e condições (F05).
abrindo	inconsistencia	so_leitura	Mostra o problema ao dono da mesa; nenhuma correção silenciosa (G03).
abrindo	servico_indisponivel	fechada	Sessão não abre; nada se perde (C02 E1).
em_cena	combate_iniciado	em_combate	Motor monta o encontro e rola iniciativa (D01).
em_cena	falha_servico	pausada	Classifica e repete; passado o limite, pausa com aviso (G05).
em_cena	falha_disco	pausada	Não aceita nova ação até gravar (G01 E1).
em_cena	encerrar	encerrando	Conclui ou marca pendente a ação em curso (C09).
em_combate	combate_encerrado	em_cena	Experiência, tesouro do módulo, fatos (D08).
em_combate	falha_servico	pausada	Turno automático da criatura antes de pausar, se for turno dela (D03 E2).
em_combate	encerrar	encerrando	Combate fica retomável no ponto exato (F05).
pausada	servico_voltou	em_cena	Retoma a ação pendente do ponto exato; nada rolado duas vezes.
pausada	encerrar	encerrando	O jogador prefere parar (G05 E1).

ESTADO ATUAL	EVENTO	NOVO ESTADO	O QUE ACONTECE JUNTO
encerrando	resumo_gravado	fechada	Resumo (G02) ou pendência de resumo registrada.
so_leitura	corrigida_pelo_dono	fechada	O dono da mesa decidiu a correção, registrada como fato novo.

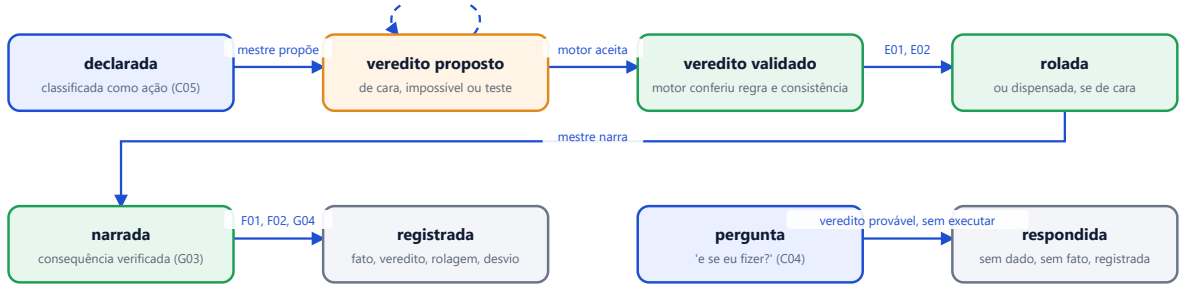


O combate. A ordem de iniciativa é dado do estado, não da máquina: a máquina só sabe de quem é a vez. 'Preso' não é estado: é um contador de rodadas sem dano no fim da rodada que, no limite, pergunta ao mestre (D08 E1).

A tabela de transições do combate

ESTADO ATUAL	EVENTO	NOVO ESTADO	O QUE ACONTECE JUNTO
montando	iniciativa_rolada	turno_pj ou turno_cri	Conforme quem é o primeiro da ordem; quem foi surpreendido não age na primeira rodada (D01).
montando	criatura_pendente	em_cena (sessão)	Combate bloqueado; o mestre narra o impasse (D01 A2).
turno_pj	declaracao_valida	turno_pj	Resolve ataque, magia ou ação do SRD; o turno segue até o jogador encerrar ou esgotar (D02).
turno_pj	declaracao_invalida	turno_pj	Recusa citando a regra; nada é gasto (D02 E1).
turno_pj	turno_encerrado	turno_cri ou turno_pj ou fim_rodada	Próximo da ordem, ou fim da rodada se era o último.
turno_cri	intencao_valida	turno_cri	Motor resolve a intenção do mestre (D03).
turno_cri	intencao_impossivel	turno_cri	Mestre escolhe outra; na insistência, ação padrão da ficha (D03 E1).
turno_cri	servico_falhou	turno_cri	Ação padrão da ficha, marcada 'turno automático' (D03 E2).
turno_cri	turno_encerrado	turno_pj ou turno_cri ou fim_rodada	Próximo da ordem.
fim_rodada	duracoes_aplicadas	turno_pj ou turno_cri	Condições expiram, salvaguardas de fim de turno, nova rodada (D06).
fim_rodada	fim_detectado	encerrado	Hostis mortos, fugidos ou rendidos; personagem morto ou fugido (D08).
fim_rodada	preso_no_limite	fim_rodada	Pergunta ao mestre, que pode declarar o fim com desvio registrado (D08 E1).

ESTADO ATUAL	EVENTO	NOVO ESTADO	O QUE ACONTECE JUNTO
qualquer	zero_pontos_de_vida	mesmo estado	Aplica inconsciência ou morte pelo caso de uso D07 sem sair do turno.



A tentativa. A pergunta tem máquina própria e curta: nunca passa por 'rolada'. É a regra RN-11 em forma de código. A ação, uma vez rolada, não volta: se o serviço cair depois da rolagem, a retomada parte de 'rolada' com o resultado já gravado (F05).

A ASSINATURA, NO PACOTE MOTOR

```
from dataclasses import dataclass
from typing import Literal

EstadoSessao = Literal["fechada", "abrindo", "em_cena", "em_combate", "pausada", "encerrando", "so_leitura"]

@dataclass(frozen=True)
class Transicao:
    proximo: EstadoSessao
    efeitos: tuple[Efeito, ...]          # descrições; quem executa é o caso de uso

class TransicaoInvalida(Exception):
    '''Par (estado, evento) fora da tabela. Nada é gravado.'''

def transicao_sessao(atual: EstadoSessao, evento: EventoSessao) -> Transicao: ...
def transicao_combate(atual: EstadoCombate, evento: EventoCombate) -> Transicao: ...
def transicao_tentativa(atual: EstadoTentativa, evento: EventoTentativa) -> Transicao: ...

# Sem banco, sem rede, sem relógio: só estado e evento. Por isso é testável
# com todos os pares da tabela, inclusive os ausentes.
```

CONCORRÊNCIA: QUEM CHEGA JUNTO

Na fase 1 há um processo e um jogador, então a única concorrência real é entre a mesa e o serviço de linguagem respondendo tarde. A proteção é simples: cada chamada ao serviço leva o identificador do turno, e uma resposta de turno já encerrado é descartada e registrada como ocorrência. O arquivo da campanha recebe uma trava de processo na abertura; uma segunda abertura da mesma campanha é recusada com a mensagem de quem a segura. A fase 2 troca a trava por fila de falas (H02), não por reescrita deste capítulo.

CAPÍTULO 5

O protocolo entre o mestre e o motor

O mestre não tem acesso ao banco, à ficha nem ao dado. Ele tem ferramentas. Cada ferramenta é uma função do motor com entrada e saída de forma fixa, validada antes de executar, registrada depois. É o único contrato do sistema que o serviço de linguagem conhece.

Convenções que valem para toda ferramenta

ASSUNTO	A REGRA
Nomes	Verbo no infinitivo, em português, minúsculo com sublinhado: consultar_cena, dar_veredito. O nome é estável; mudar o contrato cria ferramenta nova.
Entrada e saída	JSON com esquema estrito: campo desconhecido é recusado, campo obrigatório ausente é recusado, antes de qualquer regra rodar.
Identificadores	Itens canônicos, personagens e registros do diário são referenciados por identificador, nunca por nome; nome é para narrar.
Registro	Toda chamada vira um registro 'consulta' no diário, com ferramenta, entrada, saída resumida e turno. Auditoria vê o que o mestre perguntou.
Turno	Toda chamada carrega o identificador do turno. Chamada de turno encerrado é recusada e registrada.
Segredos	Toda saída que descreve o módulo vem com os segredos não descobertos separados e marcados 'não narrar', nunca misturados.
Erros	Sempre o mesmo envelope, com código estável, mensagem para o mestre e a alternativa válida quando houver. Erro de ferramenta volta ao mestre como resultado marcado, nunca como exceção que derruba o turno.
Tamanho	Saídas são recortes: a cena, não o módulo; os fatos ligados, não o diário. O limite por ferramenta é configurável e o excesso vem paginado por referência.

O ENVELOPE DE ERRO, IGUAL EM TODA FERRAMENTA

```
{
  "erro": {
    "codigo": "dificuldade_fora_da_escala",
    "mensagem": "Dificuldade 45 não existe no SRD; a escala vai de 5 a 30.",
    "alternativa": {"dificuldade_maxima": 30, "sugestao": "25 para 'quase impossível'"},
    "registro": 4812
  }
}
```

Os códigos de erro estáveis

CÓDIGO	QUANDO ACONTECE	O QUE O MESTRE DEVE FAZER
`entrada_invalida`	Campo faltando, tipo errado ou campo desconhecido.	Repetir a chamada com o esquema; não narrar nada.
`identificador_inexistente`	Cena, personagem ou item que não existe nesta versão do módulo.	Usar os vizinhos devolvidos (A05 E1).
`pericia_inexistente`	Perícia ou salvaguarda fora do SRD.	Refazer o veredito com uma da lista devolvida (E02 E1).
`dificuldade_fora_da_escala`	Dificuldade fora de 5 a 30.	Refazer dentro da escala (C05 E1).
`acao_nao_permitida`	Economia de ação, alcance, condição ou recurso impede.	Refazer; a regra citada diz o que faltou (D02 E1, E04).
`fora_do_srd`	Criatura, magia ou item não está no documento aberto.	Não inventar; o caminho é o caso de uso A04.

CÓDIGO	QUANDO ACONTECE	O QUE O MESTRE DEVE FAZER
segredo_bloqueado	A narração ou resposta revelaria segredo não descoberto.	Narrar só o que o personagem perceberia (C04 A2, G03).
contradiz_fato	O texto contradiz fato estabelecido.	Narrar de novo respeitando o fato listado (G03).
reverte_conquista	O desvio desfaz vitória, tesouro ou revelação.	Não permitido pela regra RN-04; propor outro caminho (F02 E1).
veredito_divergente	Mesma situação, veredito diferente, sem motivo.	Manter o anterior ou informar o que mudou na cena (G04).
turno_encerrado	Chamada de um turno que já acabou.	Nada; a chamada é descartada e registrada.
sem_autorizacao_envio	Pedido de texto bruto da aventura sem autorização registrada.	Nada; só o motor estrutura (DC-12).
pedido_ambiguo	Não dá para validar sem esclarecimento.	Perguntar ao jogador pela ferramenta própria (E04 E2).

As ferramentas da fase 1, nada além disso

Ferramenta nova nasce de um caso de uso com código, para o escopo continuar visível. O mestre só chama; o motor executa e registra.

FERRAMENTA	QUEM CHAMA	O QUE FAZ	O QUE DEVOLVE	CASOS
consultar_cena	mestre	Recorte da cena atual ou de outra por identificador: descrição, ligações, presentes, estado da campanha viva por cima do canônico.	cena, com segredos separados e marcados	A05 C03 C07
consultar_personagem_mestre	mestre	Ficha do personagem do mestre no módulo, estado na campanha viva, o que já disse, atitude.	personagem, com o que ele sabe e o que é segredo	C06
consultar_ficha	mestre, jogador	Ficha do personagem do jogador com o estado atual e a composição dos derivados.	ficha e, em combate, o que ainda pode fazer	B03 D02
o_que_o_personagem_sabe	mestre, jogador	Fatos conhecidos, acreditados e, para o mestre, os ocultos marcados, sobre um assunto.	conhecido, acredita, oculto	F04 C02 C04
consultar_srd	mestre, jogador	Regra, magia, criatura, item ou condição por nome ou identificador, com atribuição.	dados estruturados e texto, ou fora_do_srd	E03 B03
dar_veredito	mestre	Decide uma tentativa: de cara, impossível ou teste com perícia e dificuldade. O motor valida, compara com vereditos anteriores e, se teste, rola.	veredito aceito com o resultado da rolagem, ou erro	C05 E02 E04 G04
responder_pergunta	mestre	Registra a resposta a um 'e se eu fizer?' sem rolar nem gravar fato.	registro da consulta	C04

FERRAMENTA	QUEM CHAMA	O QUE FAZ	O QUE DEVOLVE	CASOS
registrar_fato	mestre, motor	Afirmação concreta sobre o mundo, com tipo, referência canônica e conhecimento do personagem; conferida contra os fatos ativos.	registro, ou contradiz_fato	F01 G03
registrar_desvio	mestre	Diferença em relação ao canônico, com motivo e regra invocada.	registro, ou reverte_conquista	F02
propor_caminho	mestre	Caminho alternativo quando o crítico fechou; validado contra fatos, conquistas e segredos.	aceito e ligado ao grafo, ou os motivos da recusa	F03
iniciar_combate	mestre	Participantes, zonas, quem percebeu quem. O motor monta, resolve surpresa e rola iniciativa.	ordem e estado, ou criatura pendente	D01
agir_como_criatura	mestre	Intenção da criatura no turno dela; o motor confere e resolve.	resultado mecânico para narrar, ou acao_nao_permitida	D03 D04 D05
declarar_fim_do_combate	mestre	Fim por rendição aceita, fuga ou combate preso, com desvio quando couber.	recompensa calculada e fatos	D08
perguntar_ao_jogador	mestre	Esclarecimento quando a entrada é ambígua: pergunta ou ação, qual alvo, qual das duas coisas.	a resposta do jogador	C04 E1 D02 E3 E04 E2
narrar	mestre	Saída final do turno: o texto para o jogador mais os fatos novos que ele estabelece. Passa pela verificação antes de chegar à tela.	aprovado, ou a lista de violações para narrar de novo	C03 C06 G03

DAR_VEREDITO: ENTRADA E SAÍDA REAIS

```
// entrada
{
  "turno": "t-0412",
  "tentativa": "escalar o muro do pátio",
  "veredito": {"tipo": "teste", "pericia": "atletismo", "dificuldade": 15, "modo": "normal",
               "sucesso": "chega ao alto sem ser visto", "falha": "escorrega e faz barulho"}
}

// saída: aceito e rolado
{
  "veredito": {"registro": 4813, "tipo": "teste", "pericia": "atletismo", "dificuldade": 15},
  "consistencia": {"anterior": null},
  "rolagem": {"registro": 4814, "contador": 207, "expressao": "1d20+3", "dados": [14],
              "modificadores": {"forca": 3, "proficiencia": 0}, "total": 17},
  "resultado": {"sucesso": true, "margem": 2},
  "mostrar_ao_jogador": "Atletismo: d20 (14) + 3 = 17 contra 15. Sucesso."
}

// saída: aceito, mas diverge de um veredito anterior na mesma situação
{
  "erro": {"codigo": "veredito_divergente",
           "mensagem": "Na sessão 2 a mesma tentativa foi teste de Atletismo 12.",
           "alternativa": {"manter": 12, "ou_informar": "o que mudou na cena"},
           "registro": 4815}
}
```

NARRAR: A SAÍDA FINAL DE TODO TURNO

```
{
  "turno": "t-0412",
  "texto": "Você se iça pelo muro; a argamassa cede num ponto, mas o pé encontra apoio. Do alto, o pátio: duas tochas,
um guarda cochilando.",
  "fatos": [
    {"tipo": "estado_personagem", "referencia": "loc-patio", "conhecimento": "conhecido", "resumo": "personagem no
alto do muro do pátio"},
    {"tipo": "evento", "referencia": "np-guarda", "conhecimento": "conhecido", "resumo": "guarda do pátio cochilando"}
  ],
  "desvios": []
}

// a plataforma confere texto e fatos (G03) antes de mostrar; se falhar, devolve:
{"erro": {"codigo": "segredo_bloqueado", "mensagem": "O texto menciona o túnel atrás dos barris; o personagem não o
descobriu.", "registro": 4816}}
```

O QUE NUNCA PODE ACONTECER NESTE PROTOCOLO

Um número de dado vindo do serviço de linguagem ser aceito como rolagem: o motor ignora, rola e registra a tentativa. Um segredo entrar no contexto do mestre sem a marca 'não narrar'. O texto bruto da aventura sair da máquina sem autorização registrada nesta importação. O serviço gravar estado por qualquer caminho que não seja uma ferramenta validada. Um fato ser alterado ou apagado: retcon não existe, nem pelo dono da mesa. Uma ferramenta responder com erro dentro de uma saída de sucesso: erro tem código e envelope próprios.

CAPÍTULO 6

O adaptador do serviço de linguagem

O serviço de linguagem é externo, probabilístico e trocável. O adaptador é a única parte do código que sabe falar com ele: monta o contexto da cena, oferece as ferramentas, roda o turno até a narração final e classifica cada falha. Nenhum provedor nem modelo é nomeado aqui; a bancada decide.

A INTERFACE, NO PACOTE MESTRE

```
from typing import Protocol, Callable

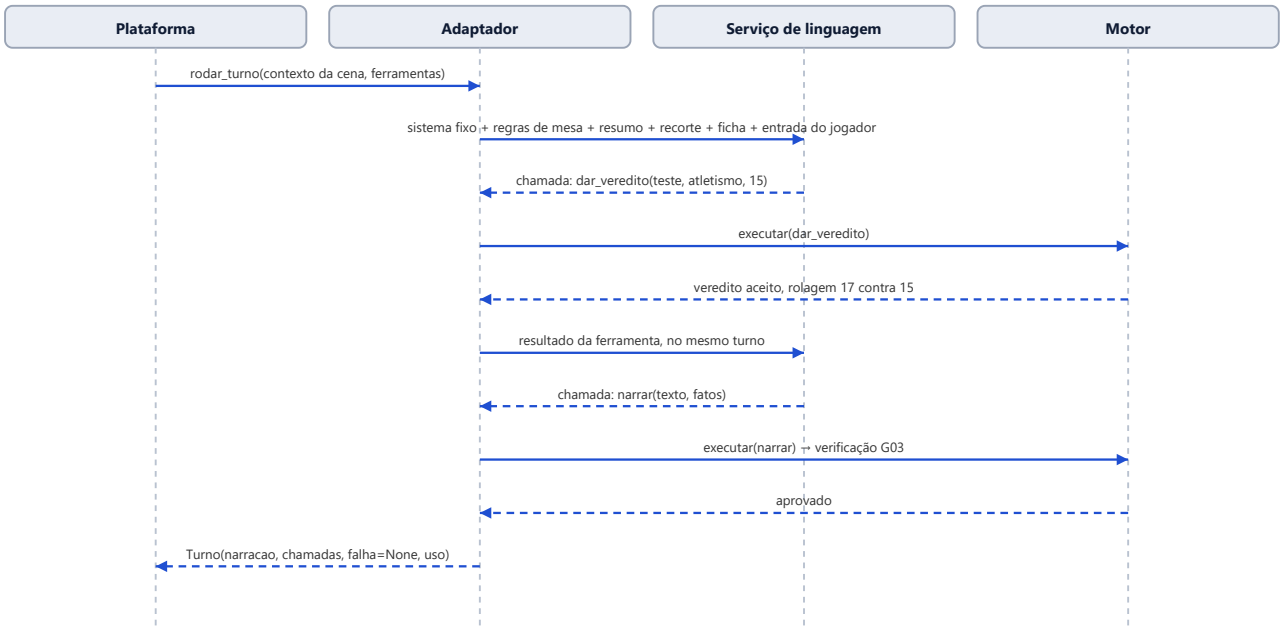
class ServicoDeLinguagem(Protocol):
    def rodar_turno(self, contexto: Contexto, ferramentas: list[Ferramenta],
                    executar: Callable[[Chamada], Resultado]) -> Turno:
        """Entrega o contexto e as ferramentas ao serviço; executa cada chamada
        pelo motor (executar) e devolve os resultados; termina quando o serviço
        produz a narração final ou quando uma falha é classificada."""

    def extrair_itens(self, trecho: Trecho, esquema: Esquema) -> ItensExtraidos:
        """Estruturação da aventura (A02): só roda com autorização registrada (DC-12);
        a saída é validada pelo esquema antes de virar rascunho."""

    def resumir(self, fatos: list[Fato], conhecimento: Conhecimento) -> Resumo:
        """Resumo de sessão (G02): só a partir dos fatos e do que o personagem sabe;
        conferido por G03 antes de ser gravado."""

@dataclass(frozen=True)
class Turno:
    narracao: Narracao | None          # saída da ferramenta narrar, já aprovada por G03
    chamadas: tuple[Chamada, ...]      # tudo o que o mestre pediu, na ordem
    falha: Falha | None                # indisponivel | formato | recusa | limite_de_saida
    uso: Uso                           # tokens de entrada e saída, cache, latência, custo estimado

# Implementações: uma por provedor, usando o SDK oficial dele; e ServicoFalso, roteirizado,
# determinístico, usado em todo teste. Nenhum teste chama a rede: a trava do pacote de testes garante.
```



Um turno de ponta a ponta. O serviço só vê recortes e ferramentas. Cada chamada volta como resultado dentro do mesmo turno; a narração é uma ferramenta como as outras, e só chega à tela depois de aprovada.

O contexto de uma cena, na ordem em que é montado

BLOCO	CONTEÚDO	MUDA QUANDO
1 · Sistema fixo	Quem o mestre é, as regras de negócio RN-01 a RN-11 em linguagem de mestre, o formato das ferramentas, o idioma.	Nunca dentro de uma campanha. É o prefixo estável que o provedor pode guardar em cache.
2 · Mesa	Classificação de conteúdo, tom, preferências (C10) e as regras de mesa da revisão (F06).	Entre sessões.
3 · Campanha	Resumo da campanha e das últimas sessões (G02, F07).	A cada sessão.
4 · Cena	Recorte da cena atual (A05) com segredos marcados, personagens presentes, o que o personagem sabe (F04).	A cada cena.
5 · Personagem	Ficha resumida e estado atual (B03); em combate, a ordem e de quem é a vez.	A cada turno.
6 · Entrada	O que o jogador disse, classificado (pergunta ou ação), e o histórico do turno em curso.	A cada mensagem.

Classificação de falha e o que fazer (G05)

FALHA	COMO SE RECONHECE	RESPOSTA DO ADAPTADOR
`indisponivel`	Erro de rede, tempo esgotado, limite de uso do provedor, erro do servidor.	Repete com espera crescente até um limite; depois devolve a falha e a sessão vai para 'pausada' com tudo salvo.
`formato`	Resposta sem a ferramenta narrar, entrada de ferramenta que não valida, texto onde devia haver chamada.	Repete uma vez pedindo o formato; persistindo, devolve a falha e o caso de uso de origem usa a alternativa mecânica.
`recusa`	O serviço declina o pedido por política de conteúdo.	Repete uma vez com o contexto reforçado (mesa de ficção, classificação configurada); persistindo, devolve a falha e registra para a bancada (DC-11).
`limite_de_saida`	A resposta foi cortada por tamanho antes de concluir.	Repete pedindo conclusão a partir do ponto; persistindo, trata como formato.
`turno_tardio`	Resposta de um turno já encerrado.	Descarta, registra ocorrência, não repete.

Cinco regras do adaptador que não são óbvias

REGRA	POR QUE EXISTE
O histórico do turno só cresce; nenhuma mensagem anterior é editada ou removida.	Provedores atuais vinculam o raciocínio interno à conversa exata; editar o passado invalida o turno e muda o comportamento. Também é o que torna o turno reproduzível na auditoria.
Todos os resultados de ferramentas pedidas numa mesma resposta voltam juntos, na ordem pedida.	Devolver em partes ensina o serviço a parar de pedir em paralelo, e quebra a correspondência entre pedido e resultado.
Erro de ferramenta volta como resultado marcado como erro, nunca como exceção.	O serviço precisa ver o código e a alternativa para refazer; uma exceção derruba o turno e perde o que já foi rolado.
Nenhuma ferramenta é forçada; o sistema fixo instrui e o esquema estrito garante a forma.	Forçar chamada não é aceito por provedores atuais e não é necessário: o motor recusa o que vier errado.
Entrada de ferramenta é analisada como JSON e validada pelo esquema antes de executar.	A serialização varia entre provedores; comparar texto cru é defeito garantido. O esquema estrito é a única fonte de verdade da forma.

O QUE AINDA NÃO SABEMOS SOBRE O PROVEDOR

Quanto custa uma cena em português com este contexto; quantos turnos cabem antes de o contexto precisar de resumo; a taxa de recusa nas cenas da classificação 'adulto'; a latência por turno com três ou quatro chamadas de ferramenta; se o provedor guarda o prefixo fixo em cache e quanto isso reduz o custo; o comportamento com esquemas estritos. O card da bancada existe para responder a essas seis perguntas com número, com a mesma bateria de cenas para cada candidato, e o resultado vira a configuração do adaptador. Até lá, o provedor falso roda tudo.

O PROVEDOR FALSO É A PEÇA QUE ECONOMIZA MAIS TRABALHO

Ele é roteirizado por cena: dado um contexto, devolve as chamadas e a narração que o roteiro manda, sem rede e sem custo. Com ele, o esqueleto ambulante, o combate, a retomada e a auditoria são testados inteiros na integração contínua, e a bancada com provedores reais vira uma comparação controlada, não uma investigação. O roteiro de cada teste é a especificação do que o mestre deveria ter feito; quando o provedor real fizer diferente, o teste diz exatamente onde. As três operações do adaptador passam por ele: o turno da mesa, a extração de itens da aventura (A02) e o resumo de sessão (G02).

O QUE VEM NAS PRÓXIMAS ENTREGAS

O resto do documento, já encomendado

Cada entrega passa pelo mesmo portão: o cliente lê, aprova ou corrige, e só então a seguinte começa.

Entrega 3	Capítulos 7 e 8: três fluxos de ponta a ponta e o roteiro em cards, com critério de aceite por card e dependências, sem estimativa de tempo.
Entrega 4	Capítulo 9 e fechamento: ambientes, níveis de teste, definição de pronto, o que fazer primeiro, o que este documento assume e as lacunas do Mapa tocadas.